

(Task 1 of 7) In this tutorial, we will learn *more* about using functions as values.

0

(Task 2 of 7) What is the result of running this program?

Lispy | Lispy [Run 

Python

```
(defvar x 1)      x = 1
(deffun (f)       def f():
  (deffun (addx y)   def addx(y):
    (+ x y))         return x + y
  addx)             return addx
(defvar g (f))    g = f()
(set! x 2)        x = 2
(g 0)             print(g(0))
```

1

2

The answer is 2.

3

▼ Textual explanation

You are right that `x` is bound to 1 when `addx` is created. You might think the function remembers the value 1. However, `addx` does not remember the value of `x`. Rather, it remembers the environment and hence always refers to the latest value of `x`. In SMoL, functions refer to the latest values of variables defined outside their definitions.

Click [here](#) to run this program in the Stacker.

What is your thought? (Feel free to skip this question.)

4

(You skipped the question.)

5

What is the result of running this program?

Lispy | Lispy [Run 

JavaScript

```
(defvar a 2)      let a = 2;
(deffun (make)    function make() {
  (deffun (f b)    function f(b) {
    (+ a b))      return a + b;
  f)              }
  return f;
(defvar g (make)) }
(set! a 1)        let g = make();
(g 1)             a = 1;
                  console.log(g(1));
```

2

7

You predicted the output correctly 🎉🎉🎉

8

(Task 3 of 7) What is the result of running this program?

Lispy | Lispy [Run 

Pseudo

```

(deffun (foo)
  (defvar n 0)
  (deffun (bar)
    (set! n (+ n 1))
    n)
  bar)
(defvar f (foo))
(defvar g (foo))

(f)
(g)
(f)

```

```

fun foo():
  let n = 0
  fun bar():
    n = n + 1
    return n
  end
  return bar
end

let f = foo()
let g = foo()
print(f())
print(g())
print(f())

```

1 2 3

10

The answer is 1 1 2.

11

▼ Textual explanation

`foo` is called twice. The first call binds `n` to `0`. The second call also binds `n` to `0`. You might think the second binding applies everywhere. However, it only applies to the second call. `(f)` mutates the first binding, while `(g)` mutates the second. In SMoL, variable references follow the hierarchical structure of blocks.

Click [here](#) to run this program in the Stacker.

What is the result of running this program?

Lispy | Lispy [Run 

JavaScript

```

(deffun (f)
  (defvar n 1)
  (deffun (dbl)
    (set! n (* n 2))
    n)
  dbl)
(defvar dbl1 (f))
(defvar dbl2 (f))

(dbl1)

```

```

function f() {
  let n = 1;
  function dbl() {
    n = n * 2;
    return n;
  }
  return dbl;
}

let dbl1 = f();
let dbl2 = f();

```

```
(dbl2)
(dbl1)

console.log(dbl1());
console.log(dbl2());
console.log(dbl1());
```

2 2 4

13

You predicted the output correctly 🎉🎉🎉

14

(Task 4 of 7) What is the result of running this program?

Lispy | 

Lispy [Run 

Scala 3

```
(deffun (bar y)
  (deffun (add x)
    (+ x y))
  addy)
(defvar f (bar 2))
(defvar g (bar 4))
(f 2)
(g 2)

def bar(y : Int) =
  def addy(x : Int) =
    x + y
  addy
val f = bar(2)
val g = bar(4)
println(f(2))
println(g(2))
```

4 6

16

You predicted the output correctly 🎉🎉🎉

17

The value of (bar 2) is the function addy defined in an environment where y is bound to 2. The value of (bar 4) is *another* addy defined in an environment where y is bound to 4. The two addy functions are *different* values. So, the value of (f 2) is 4, while the value of (g 2) is 6.

Click [here](#) to run this program in the Stacker.

(Task 5 of 7) What did you learn about functions from these programs?

18

binding

19

(Task 6 of 7) Functions remember the environment in which they are defined. That is, function bodies are "enclosed" by the environments in which the function values are created. So, function values are called **closures**.

20

Any feedback regarding these statements? Feel free to skip this question

21

Any feedback regarding these statements. Feel free to skip this question.

(You skipped the question.)

22

(Task 7 of 7) Please scroll back and select 1-3 programs that make the point that

23

Functions remember the environment in which they are defined.

You don't need to select *all* such programs.

(You selected 1 programs)

24

Okay. How does this program (15) support the point?

25

h

26

Let's review what we have learned in this tutorial.

27

Functions remember the environment in which they are defined. That is, function bodies are "enclosed" by the environments in which the function values are created. So, function values are called **closures**.

You have finished this tutorial 🎉🎉🎉

Please [print](#) the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1781419918670